

Shiraz University

Department of Computer Science & Engineering

Text Mining & Natural Language Processing

Assignment 1

Context-Aware Noisy Text Restoration with N-gram Language Models

Instructor: Dr. Mostafa Fakhrahmad

Teaching Assistants: Alireza Shayan, Hossein Karami

Course: Text Mining & NLP

Term: Spring 1405 / 2026

Start: Tuesday, May 26, 2026 / 5 Khordad 1405

Deadline: 23:59 of Tuesday, June 16, 2026 / 26 Khordad 1405

Contents

1 Overview & Learning Goals **2**
- 1.1 Core Objective . . . 2
- 1.2 Learning Goals . . . 2

2 Provided Data **2**
- 2.1 File Structure . . . 2
- 2.2 Data Usage Policy . . . 3
- 2.3 Noise Types . . . 3

3 Scope & Allowed Methods **4**
- 3.1 N-gram Requirement . . . 4
- 3.2 Permitted Auxiliary Methods . . . 4
- 3.3 AI Tool Policy . . . 4

4 Required System Components **5**
- 4.1 Language Modelling . . . 5
- 4.2 Candidate Generation . . . 6
- 4.3 Candidate Scoring . . . 7
- 4.4 Abstain / No-Edit Decision . . . 7
- 4.5 Baselines to Implement . . . 8

5 Standard CLI Interface & Code Structure **8**
- 5.1 Command Line Interface . . . 8
- 5.2 Recommended Code Structure . . . 9

6 Required Output Files **10**
- 6.1 `dev_predictions.csv` . . . 10
- 6.2 `test_predictions.csv` . . . 10
- 6.3 `dev_metrics.json` . . . 10
- 6.4 `summary.json` . . . 11
- 6.5 `trace.jsonl` . . . 11
- 6.6 `report.pdf` . . . 11
- 6.7 `results/extra/` . . . 11

7 trace.jsonl — Full Specification **12**
- 7.1 Purpose . . . 12
- 7.2 Top-Level Schema . . . 12
- 7.3 token_decisions Schema . . . 12
- 7.4 Candidate List Policy . . . 13
- 7.5 Abstention Example . . . 13

8 Report Requirements **14**

9 Grading **15**

10 Submission **15**

11 Key Hints **16**

1. Overview & Learning Goals

1.1. Core Objective

In this assignment, you will build a **context-aware noisy text restoration system** that uses N-gram language modeling as a central component. The system must make three kinds of decisions:

- **When to edit:** Identify tokens that are likely corrupted.
- **What to edit:** Choose the best correction from a set of candidates.
- **When not to edit:** Abstain from making a change if the evidence is insufficient.

The emphasis is on **local context sensitivity** and **avoiding unnecessary changes**. A system that blindly corrects everything is penalised. A system that thoughtfully abstains when uncertain is rewarded.

1.2. Learning Goals

By completing this assignment, students will:

1. Build and analyse N-gram language models (unigram, bigram, trigram) from raw text.
2. Use N-gram probabilities to score token corrections in context.
3. Handle correction ambiguity through candidate ranking and confidence thresholds.
4. Evaluate restoration quality using standard NLP metrics.
5. Perform ablation studies to justify design choices.
6. Produce fully reproducible, well-documented outputs.

2. Provided Data

2.1. File Structure

```
HW1_data/
|-- train_clean.txt    <- clean text for N-gram model training
|-- dev_clean.txt      <- clean reference sentences (development
   set)
|-- dev_noisy.txt      <- noisy version of dev set (for tuning and
    error analysis)
+-- test_noisy.txt     <- blind test set (no reference provided)
```

Listing 1 – *Dataset directory layout*

All files are UTF-8 encoded, one sentence per line.

2.2. Data Usage Policy

- `train_clean.txt` is **exclusively** for N-gram estimation. Do not use it to check "correct" spellings directly.
- `dev_clean.txt` and `dev_noisy.txt` are for system tuning, threshold selection, and understanding error patterns.
- `test_noisy.txt` is the blind test set. **Manual correction of any test sentence is prohibited.**
- The dev set may be used for hyperparameter tuning, but results must be reported honestly — do not overfit to dev.

2.3. Noise Types

The noisy text was generated by applying one or more of the following transformations to clean tokens:

Table 1 – *Noise transformation types with examples*

Type	Example	Description
Character substitution	`teh` → `the`	One character replaced by another
Character deletion	`wrld` → `world`	A character is removed
Character insertion	`helllo` → `hello`	An extra character is added
Character doubling	`bbook` → `book`	A character is repeated
Transposition	`recieve` → `receive`	Two adjacent characters are swapped
Case error	`london` → `London`	Incorrect capitalisation
No change	—	The token is already correct

★ Important Note

Multiple noise types per token: A single token may have been corrupted by *more than one* transformation simultaneously. For example, `Wrold` may contain both a *transposition* (`rl` → `rl`) and a *case error*. Your candidate generation and scoring pipeline must not assume that each token has at most one noise type applied to it.

★ Important Note

Not every unusual or rare token is an error. The system must **not** assume that low-frequency words are wrong. Overcorrection of valid rare words is a common and **penalised** mistake.

3. Scope & Allowed Methods

3.1. N-gram Requirement

N-gram language models must serve as the **primary scoring mechanism** in the final system. Specifically:

- The final correction decision for each token must be based on N-gram scores (e.g., contextual log-probability).
- Replacing or bypassing N-gram scoring with a *dictionary lookup* or *neural model output* is **not permitted**.
- The contribution of the N-gram component must be demonstrated via ablation.

3.2. Permitted Auxiliary Methods

The following may *support*—but not replace—the N-gram scoring component:

Table 2 – *Allowed auxiliary methods*

Role	Examples
Candidate generation	Edit distance, keyboard neighbours, OCR rules
Candidate filtering	Dictionary / lexicon membership
Score adjustment	Unigram prior, edit penalty, confidence margin
External tools	Neural models, pretrained embeddings, AI assistants

▹ Note

Rule of thumb: If removing the N-gram component leaves the system functionally unbroken, the requirement is not met.

3.3. AI Tool Policy

AI tools (chatbots, coding assistants, etc.) are **allowed without restriction**. However:

- Students must be able to **fully explain** any code or design choice in their submission.
- AI usage must be **disclosed** in the report (Section 10 of the report).
- Inability to explain submitted work during oral evaluation will result in grade penalties.

4. Required System Components

4.1. Language Modelling

Implement and evaluate three N-gram model orders: **unigram, bigram, and trigram**. At least one model of order ≥ 2 must be used in the final restoration system.

For each model, document the following design decisions:

Tokenisation

Describe how raw text is split into tokens. Consider punctuation handling, whitespace normalisation, and special characters. Be consistent between training and inference.

Sentence Boundaries

Each line in the dataset corresponds to exactly one sentence. Decide whether to add explicit boundary markers (e.g., `<s>` and `</s>` tokens) at the start and end of each sentence, and document how this choice affects N-gram counts at sentence edges.

Case Handling

Decide whether to lowercase all tokens, preserve case, or use a mixed strategy. Document the choice and its effect on vocabulary size and unknown-word rate.

Unknown Words

Define how out-of-vocabulary (OOV) tokens are handled. Common strategies:

- Replace all tokens with frequency below a threshold k with a special `<UNK>` token during training.
- At inference, map unseen tokens to `<UNK>` before scoring.

Smoothing

Raw N-gram counts will assign zero probability to unseen N-grams. Choose and implement a smoothing method. Common options:

- **Laplace (add-1) smoothing:**

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|} \tag{1}$$

- **Add-k smoothing:** Replace 1 in Eq. (1) with a smaller constant $k < 1$.
- **Stupid backoff or Kneser-Ney:** More sophisticated options for better generalisation.

Evaluation of Language Models

Compare models using **perplexity** on the dev set:

$$\mathrm{PP}(W) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i \mid \text{context})\right) \tag{2}$$

Lower perplexity = better model. Report perplexity for all three orders and discuss the trade-off between model order and data sparsity.

4.2. Candidate Generation

For each token in a noisy sentence, generate a set of **candidate corrections**. The original token must always be included as a candidate (representing the "no change" option).

Candidate generation strategies to consider:

Table 3 – *Candidate generation strategies*

Strategy	Description
Edit distance 1	All strings within 1 edit (insert, delete, substitute, transpose)
Edit distance 2	Broader but noisier candidate set
Keyboard neighbours	Characters adjacent on a QWERTY keyboard
Case normalisation	Lowercase / uppercase / title-case variants
Dictionary filtering	Keep only candidates appearing in a known vocabulary
Confusion rules	Domain-specific substitution pairs (e.g., OCR: `0`↔`o`, `1`↔`l`)

▹ **Note**

Practical considerations:

- Edit distance 1 over a large vocabulary can produce hundreds of candidates per token. Filtering by dictionary membership is *strongly recommended*.
- Generating candidates for every token is expensive. Consider limiting candidate generation to tokens that look suspicious (e.g., not in vocabulary, low unigram frequency).
- The candidate set size directly affects runtime. Document your choices.
- **Short token caution:** For tokens of length 1 or 2 (e.g., `a`, `in`), edit distance 1 can generate an extremely large candidate set (potentially covering a significant fraction of the vocabulary). It is recommended to either *skip candidate generation* for such tokens entirely, or apply a stricter dictionary filter before scoring. Document your chosen policy.

4.3. Candidate Scoring

Each candidate must be assigned a score that reflects how well it fits in context. The N-gram model must be a meaningful part of this score.

Basic contextual score (bigram)

$$\text{score}(c) = \log P(c \mid w_{i-1}) + \log P(w_{i+1} \mid c) \tag{3}$$

Trigram extension

$$\text{score}(c) = \log P(c \mid w_{i-2}, w_{i-1}) + \log P(w_{i+1} \mid w_{i-1}, c) + \log P(w_{i+2} \mid c, w_{i+1}) \tag{4}$$

Additional score components (optional but recommended)

- **Unigram prior:** $\log P(c)$ — favours common words.
- **Edit penalty:** $-\lambda \cdot \text{edit_distance}$ — penalises large changes.
- **Confidence margin:** The gap between the top-1 and top-2 candidate scores.
- **Bidirectional score:** Average of left-context and right-context scores.

Final combined score

$$\text{score}_{\text{total}}(c) = \alpha \cdot \text{score}_{\text{context}}(c) + \beta \cdot \log P(c) + \gamma \cdot \text{edit_penalty}(c) \tag{5}$$

where α, β, γ are tunable weights. Document your chosen values and how you tuned them.

4.4. Abstain / No-Edit Decision

The system must be able to **leave a token unchanged** when the evidence for correction is not strong enough. This is implemented via a threshold τ:

$$\text{score}(\hat{c}) - \text{score}(\text{original}) > \tau \tag{6}$$

where $\hat{c}$ is the best non-original candidate.

Threshold tuning

- τ too *low* $\Rightarrow$ system edits too aggressively $\Rightarrow$ high overcorrection rate.
- τ too *high* $\Rightarrow$ system misses real errors $\Rightarrow$ low recall.
- Tune τ on the dev set by sweeping values and observing the trade-off between edit F1 and overcorrection rate.

▹ **Note**

Sentence-level abstention (optional): In addition to token-level abstention, you may implement a sentence-level decision: if no token in a sentence has a margin above τ, output the sentence unchanged and set `"edited": false`.

4.5. Baselines to Implement

You must implement the following three baselines to compare against your final system:

Table 4 – *Required baseline systems*

Baseline	Description
No-change	Always return the input sentence unchanged.
Unigram correction	For each token that is *out-of-vocabulary* (OOV), replace it with the highest-frequency in-vocabulary candidate within edit distance 1. In-vocabulary tokens are left unchanged. **No abstention threshold is applied.**
Contextual N-gram	Use bigram or trigram context for scoring, with abstention.

▹ **Note**

The **Unigram correction** baseline operates only on OOV tokens. Applying it to all tokens (including in-vocabulary ones) would result in aggressive overcorrection and is *not* the intended behaviour. This design choice also clarifies the ablation: the gap between Unigram correction and Contextual N-gram reflects the benefit of context over frequency alone.

5. Standard CLI Interface & Code Structure

5.1. Command Line Interface

The entire pipeline must be runnable via a single command:

```
python system.py \
    --train      HW1_data/train_clean.txt \
    --dev-noisy  HW1_data/dev_noisy.txt   \
    --dev-clean  HW1_data/dev_clean.txt   \
    --test-noisy HW1_data/test_noisy.txt  \
    --outdir     results
```

Listing 2 – *Main CLI invocation*

Minimum required behaviour:

- End-to-end execution without manual intervention.
- All output files written to `--outdir`.
- Reproducible results (fix random seeds if any randomness is used).

Recommended optional arguments:

```
--seed 42             # random seed for reproducibility
--config config.json  # load hyperparameters from a JSON file
--ngram-order 3       # override default N-gram order
--threshold 0.5       # override abstain threshold
```

Listing 3 – *Optional CLI arguments*

5.2. Recommended Code Structure

Although the entry point is `system.py`, you are **strongly encouraged** to organise your implementation across multiple modules rather than placing all code in a single file. A suggested layout is:

```
project_root/
|-- system.py            <- entry point (CLI argument parsing +
   pipeline call)
|-- ngram_model.py       <- N-gram counting, smoothing,
   probability queries
|-- candidates.py        <- candidate generation (edit distance,
   case variants)
|-- scorer.py            <- candidate scoring and abstain
   decision
|-- evaluator.py         <- metric computation and output file
   writing
|-- config.json          <- hyperparameters (optional but
   recommended)
+-- results/
    |-- dev_predictions.csv
    |-- test_predictions.csv
    |-- dev_metrics.json
    |-- summary.json
    |-- trace.jsonl
    |-- report.pdf
    +-- extra/           <- optional
```

Listing 4 – *Recommended project structure*

▹ **Note**

Module responsibilities:

- `system.py` — argument parsing only; imports and calls the other modules.
- `ngram_model.py` — `NgramModel` class: `train()`, `log_prob()`, `perplexity()`.
- `candidates.py` — `generate_candidates(token, vocab)` using edit distance and case normalisation.
- `scorer.py` — `score_candidates()` with N-gram context, unigram prior, edit penalty, and abstain threshold logic.
- `evaluator.py` — `compute_metrics()` and writers for output files.

You may rename or split these modules, but the boundaries above reflect a clean separation of concerns. All modules must be importable without side effects.

6. Required Output Files

All output files must be written to the `--outdir` directory (default: `results/`).

6.1. `dev_predictions.csv`

One row per sentence in the dev set.

Table 5 – *Schema for* `dev_predictions.csv`

Column	Type	Description
`sentence_id`	int	0-indexed sentence number
`input_text`	string	Original noisy sentence
`predicted_text`	string	System output
`reference_text`	string	Clean reference
`edited`	bool	Whether any token was changed

6.2. `test_predictions.csv`

Same format as `dev_predictions.csv`, without the `reference_text` column.

6.3. `dev_metrics.json`

A JSON object containing all evaluation metrics computed on the dev set:

Listing 5 – *Example* `dev_metrics.json`

```
{
        "sentence_accuracy"  : 0.72,
        "token_accuracy"     : 0.94,
        "WER"                : 0.08,
        "CER"                : 0.03,
        "edit_precision"     : 0.81,
        "edit_recall"        : 0.76,
        "edit_F1"            : 0.78,
        "overcorrection_rate": 0.05
}
```

Metric definitions:

- **Sentence accuracy:** Fraction of sentences where prediction exactly matches reference.
- **Token accuracy:** Fraction of tokens where prediction matches reference.
- **WER (Word Error Rate):**

$$\text{WER} = \frac{\text{substitutions} + \text{insertions} + \text{deletions}}{\text{total reference tokens}}$$

- **CER (Character Error Rate):** Same as WER but at character level.
- **Edit precision:** Of all tokens the system changed, what fraction were actually wrong in the input?

- **Edit recall:** Of all tokens that were wrong in the input, what fraction did the system fix?
- **Edit F1:** Harmonic mean of edit precision and recall.
- **Overcorrection rate:** Fraction of tokens the system changed that were *already correct* in the input. Formally:

$$\text{overcorrection rate} = \frac{\#\text{ tokens changed that were already correct}}{\#\text{ tokens changed by the system}}$$

  A value of 0 means no correct token was ever touched; a value of 1 means every edit was harmful.

6.4. `summary.json`

System configuration and metadata:

Listing 6 – *Example* `summary.json`

```
{
  "student_name"       : "...",
  "student_id"         : "...",
  "ngram_order"        : 3,
  "smoothing"          : "laplace",
  "tokenization"       : "whitespace",
  "case_handling"      : "lowercase",
  "unk_threshold"      : 2,
  "abstain_threshold"  : 0.5,
  "candidate_strategy" : "edit_distance_1 + dictionary",
  "score_weights"      : {"context": 0.7, "prior": 0.2, "edit_penalty":
      0.1},
  "random_seed"        : 42
}
```

6.5. `trace.jsonl`

Detailed per-sentence decision log. See Section 7 for full specification.

6.6. `report.pdf`

Written report. See Section 8 for full specification.

6.7. `results/extra/`

Optional directory for additional outputs: plots, intermediate files, ablation logs, etc.

7. trace.jsonl — Full Specification

This is the most important output file for interpretability and debugging. Each line is a valid JSON object representing one sentence.

7.1. Purpose

The trace file must answer the following questions for every sentence:

- What was the input and what was the output?
- Which token positions were examined?
- What candidates were considered at each position?
- Why was a particular candidate selected (or rejected)?
- Why did the system abstain at a given position?

7.2. Top-Level Schema

Listing 7 – *Top-level* `trace.jsonl` *schema*

```
{
  "sentence_id" : 0,
  "input_text"  : "the qick brown fox",
  "output_text" : "the quick brown fox",
  "edited"      : true,
  "sentence_level": {
    "input_score"        : -12.4,
    "output_score"       : -10.1,
    "score_improvement"  : 2.3,
    "decision"           : "edit",
    "reason"             : "token 'qick' replaced by 'quick' with
       margin 2.3 > threshold 0.5"
  },
  "token_decisions": [...],
  "summary": {
    "positions_considered": 4,
    "edits_made"          : 1,
    "abstentions"         : 3
  }
}
```

7.3. token_decisions Schema

Each element of `token_decisions` corresponds to one token position:

Listing 8 – *Per-token decision schema*

```
{
  "token_index"       : 1,
  "original"          : "qick",
  "final"             : "quick",
  "changed"           : true,
  "candidates": [
    {"text": "quick", "score": -2.1, "rank": 1},
    {"text": "qick",  "score": -4.4, "rank": 2},
```

```
    {"text": "kick",  "score": -5.0, "rank": 3}
  ],
  "selected_candidate": "quick",
  "score_original"    : -4.4,
  "score_selected"    : -2.1,
  "score_margin"      : 2.3,
  "threshold"         : 0.5,
  "decision"          : "edit",
  "decision_reason"   : "margin 2.3 exceeds threshold 0.5; best
     candidate 'quick' preferred"
}
```

7.4. Candidate List Policy

- The original token **must always appear** in the candidate list.
- Candidates must be **sorted by score, best first**.
- Scores must be **JSON numbers** (not strings). Use log-probabilities or any consistent numeric scale — but be consistent throughout the file.
- If a position was examined but the system abstained, `changed` must be `false` and `decision` must be `"abstain"`.
- An empty `token_decisions` array is only valid if **no token-level exploration was performed at all** for that sentence.

7.5. Abstention Example

Listing 9 – *Example of an abstention entry*

```
{
  "token_index"       : 0,
  "original"          : "the",
  "final"             : "the",
  "changed"           : false,
  "candidates": [
    {"text": "the", "score": -1.2, "rank": 1},
    {"text": "teh", "score": -6.8, "rank": 2}
  ],
  "selected_candidate": "the",
  "score_original"    : -1.2,
  "score_selected"    : -1.2,
  "score_margin"      : 0.0,
  "threshold"         : 0.5,
  "decision"          : "abstain",
  "decision_reason"   : "original token is already top-ranked; margin
     0.0 below threshold"
}
```

8. Report Requirements

The report (`report.pdf`) must be a self-contained document covering the following sections:

1. **Problem Setting:** Why is noisy text restoration harder than standard spell checking? What makes context necessary?
2. **Method:** Full pipeline description — from raw noisy input to final output. Include a diagram if helpful.
3. **Language Model Design:** N-gram orders evaluated, smoothing method, tokenisation, case handling, unknown-word strategy, perplexity results.
4. **Restoration Design:** Candidate generation strategy, scoring function (with formula), abstain rule, threshold tuning procedure.
5. **Experimental Settings:** All configurations tested, hyperparameter ranges, how dev set was used.
6. **Results:** Metrics table matching `dev_metrics.json`. Compare your system against all three baselines.
7. **Ablation Study:** At least **3 meaningful comparisons**, each isolating one design decision. Examples:
   - Bigram vs. trigram context scoring
   - With vs. without edit penalty
   - With vs. without abstention (threshold = 0 vs. tuned τ)
   - With vs. without dictionary filtering
   - Laplace smoothing vs. add-k smoothing
8. **Error Analysis:** At least **5 concrete failure cases** from the dev set:
   - ≥ 2 cases where the system made a **wrong edit** (changed a correct token)
   - ≥ 2 cases where the system **missed a real error** (left a wrong token unchanged)
   - ≥ 1 other interesting case

   For each case: show noisy input / system prediction / reference, and diagnose the root cause.
9. **Reflection:** One systematic weakness of your system, and one concrete improvement you would make with more time.
10. **AI Usage Disclosure:** Describe how AI tools and their specific versions were used (e.g., "used Gemini 3 Pro for boilerplate code," "used ChatGPT-5 to debug smoothing formula," "used Claude 4.6 Sonnet for brainstorming," or similar usage with GapGPT, Grok, Kimi, etc.). This section is for transparency, not penalisation.

9. Grading

Table 6 – *Grading rubric*

Component	Weight	What is Evaluated
Hidden test restoration quality	30%	Metrics on `test_predictions.csv`
Dev-set quality and reproducibility	20%	Metrics on dev + clean re-run
Overcorrection control	15%	Overcorrection rate on dev and test
Ablation and error analysis	15%	Depth and rigour of report sections 7–8
`trace.jsonl` quality	10%	Completeness, correctness, interpretability
Code clarity and format compliance	10%	Readable code, correct output format
Total	**100%**	

▹ Note

Grading is relative to class performance. A system that avoids harmful edits, explains its choices, follows the output structure, and shows systematic experimentation is valued over a complex black-box with better raw numbers.

10. Submission

★ Important Note

Submit a single zip archive named:

`NLP-HW1-FirstName_LastName-StudentNumber.zip`

The archive must contain all your code files (following the structure recommended in Section 5), along with the `results/` directory containing all required outputs and your PDF report.

11. Key Hints

- **Rare words are not always wrong.** A low-frequency token in the training data is not evidence of noise. Resist the urge to "fix" valid rare words.
- **Overcorrection is often worse than under-correction.** Missing a few real errors is less damaging than introducing new ones. Tune τ accordingly.
- **When candidates are close in score, abstain.** A small margin means the model is uncertain. Uncertainty should favour the original.
- **The trace file is your best debugging tool.** Before analysing metrics, read a few trace entries manually to understand what the system is actually doing.
- **Simple baselines justify complexity.** If your full system barely outperforms the no-change baseline, something is wrong. Check your scoring function and threshold.
- **Reproducibility matters.** Fix all random seeds. The graders will re-run your system.